



1 - Introduction

This section provides an overview of the most important Vulkan concepts. It includes information that is already available in the official documentation and other resources. Observe that this section can be quite dense with information, and may contain some references that require some prior knowledge. Therefore, I also included basic computer graphics concepts (rendering pipeline, stages, shaders, etc.) to help keep this section as self-contained as possible. However, if you still have doubts, I recommend coming back to this section after going through the next tutorial, where general computer graphics concepts will be explained in detail from scratch.

1.1 - What is Vulkan?

Vulkan is an API Specification created and maintained by The Khronos Group, an open consortium of leading hardware and software companies. The Vulkan specification outlines a low-level API that is designed to expose the GPU to application developers with a minimal level of abstraction provided by the device driver. This allows Vulkan applications to take advantage of lower CPU overhead, reduced memory usage, and increased performance stability.

For a GPU to be compatible with Vulkan, it requires a hardware vendor-supplied driver that maps Vulkan API calls to the hardware, which of course must be capable of executing the corresponding operations. NVIDIA, AMD, and Intel offer their Vulkan implementations through driver update packages for a variety of GPU architectures and platforms, including Windows, Linux, and Android.

The Khronos Group provides C99 header files that are derived from the Vulkan Specification, and can be used by developers to interface with a hardware-specific implementation of the Vulkan API. Additionally, multiple language bindings are available for developers who do not work with C code.

1.2 - Why Vulkan?

You may be wondering why The Khronos Group didn't just release a new version of the OpenGL API and why you should choose Vulkan over OpenGL.

To answer the first question, the fact is that after the launch of Vulkan, a new version of OpenGL was indeed released. However, OpenGL and Vulkan offer the same functionality but with different levels of abstraction in their communication with the GPU. The differences in design between OpenGL and Vulkan are so significant that updating OpenGL without fundamentally changing it was nearly impossible. Hence, Vulkan was created.

Regarding the second question, Vulkan is a low-level API that provides applications with a lot of power, but in return it also requires applications to take on a lot of responsibility in ensuring proper functionality. Before you begin learning or working with Vulkan, it's important to evaluate its pros and cons to determine if it's the right choice for your projects.

1.2.1 - Advantages

Despite being more difficult to learn and use, Vulkan still brings some advantages compared to OpenGL, as outlined in the following sections.

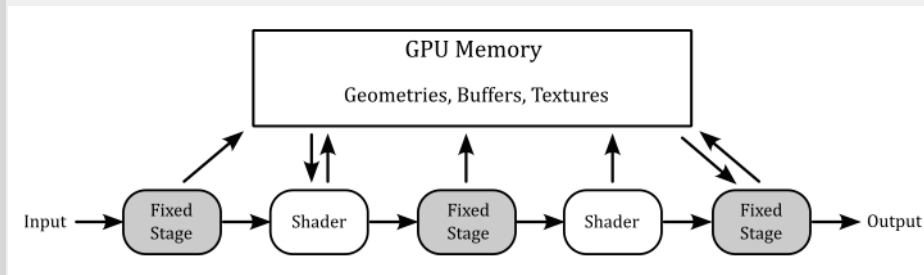
State management

OpenGL uses a single global state which is known at draw time, meaning that it can be challenging and/or costly to implement certain optimizations.

The term "state" refer to the set of information the GPU needs to execute its work. It specifies the behaviour\setup of every stage in the pipeline when we are going to draw something: shader code (that is, the code the GPU executes for programmable stages), input and output resources (of the pipeline stages), primitive topology (that is, if you want to draw points, lines, or triangles), etc.

Graphics operations are performed through a rendering pipeline composed of stages. Some of these stages are called shaders and are programmable, which means we can set the tasks performed by writing special programs (shader code) to run on GPU cores for programmable stages.

Other stage are fixed, which means they perform predefined operations. However, we can still configure fixed stages to set how they should perform their tasks.



Each stage outputs its results as input of the next stage. Additionally, memory resources (such as buffers and textures) can be accessed in GPU memory by pipeline stages to perform their tasks. However, if programs executed by programmable stages make use of memory resources, we need to bind resource variables in the shader code to actual resources in memory.

Additional details will be provided in the next tutorial.

Vulkan uses pipeline objects to store used states ahead of time, allowing more predictable application of shader-based optimizations and reducing runtime costs.

This difference results in a notable decrease in the CPU overhead of graphics drivers, but it requires applications to determine the necessary states upfront to construct the state objects and benefit from the reduced overhead.

API execution model

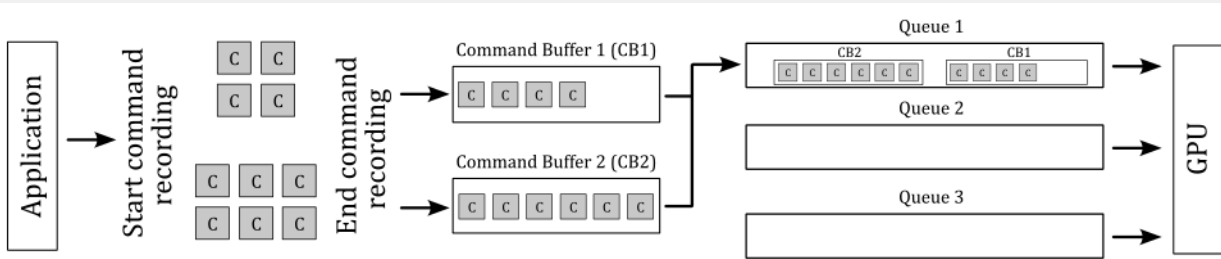
OpenGL uses a synchronous rendering model, which implies that each API call has to act as if all prior API calls have already been executed. However, in practice no modern GPU works in this way, as rendering workloads are processed asynchronously. The synchronous model is just an illusion maintained by the device driver. To maintain this illusion, the driver has to monitor which resources are being read or written by each rendering operation, guarantee that the workloads are carried out in a valid order to avoid rendering issues, and ensure that API calls which need a data resource wait until that resource is safely available.

Vulkan uses an asynchronous rendering model that mirrors how the modern GPUs work. Applications queue rendering commands into a queue, manages the execution order of workloads through explicit scheduling dependencies, and explicitly synchronize access to resources.

This difference results in a notable decrease in the CPU overhead of graphics drivers, but it requires applications to handle command recording and queuing, and resource synchronization.

GPUs have queues to store list of commands called command buffers submitted by applications. The command buffer is the GPU unit of execution. That is, a GPU doesn't execute single commands sent from the application, as suggested by the synchronous model exposed by OpenGL. On the other hand, the execution model of Vulkan is asynchronous in the sense that CPU (through a Vulkan application) and GPU have different timelines (that is, the time when they execute something). For example, a Vulkan application can record graphics commands into command buffers and submitting them into a queue (CPU timeline). Then, this queue is read by the device (GPU) that executes the

command buffers (GPU timeline). Also, several command buffers can be built simultaneously in parallel using multiple threads in an application. The following diagram shows a simplified representation of this execution model.



In the image above, the application records two command buffers containing several commands. These commands are then submitted to one or more queues depending upon the workload nature. The device accesses its queues to process command buffer workloads and either displays them on the output display or returns them to the application for further processing.

API threading model

OpenGL uses a single-threaded rendering model, which greatly restricts applications from using multiple CPU cores to parallelize rendering operations.

Vulkan uses a multi-threaded rendering model, which allows applications to parallelize rendering operations across multiple CPU cores.

This difference allow Vulkan applications to benefit from systems with a multi-core CPU.

API error checking

OpenGL use extensive run-time error checking which increases driver overheads in all applications.

Vulkan does not require the driver to implement runtime error checking. This implies that incorrect use of the API may result in rendering corruption or application crashes. Instead of the constant error checking of OpenGL, Vulkan offers a framework that permits layer drivers to be placed between the application and the native Vulkan driver. These layers can introduce error checking and other debugging functionality and have a significant advantage since they can be removed when not needed.

This difference results in a notable decrease in the CPU overhead of graphics drivers, at the expense of making many errors undetectable unless a layer driver is used.

Memory allocation

OpenGL uses a client-server memory model. This means the application (the client) cannot directly allocate or manage the memory backing GPU (the server) resources. The driver manages all of these resources individually using internal memory allocators, and synchronizes resources between client and server.

Vulkan is designed to give the application more direct control over memory resources, how they are allocated, and how they are updated.

This difference results in a notable decrease in the CPU overhead of graphics drivers, and gives the application more control over memory management. The application can further reduce CPU load: for example, by grouping objects with the same lifetime into a single allocation and tracking them collectively instead of tracking them individually.

Memory usage

OpenGL uses a typed object model, which tightly couples a logical resource with the physical memory which backs it. While this

model is easy to use, it prevents reusing the same physical memory for different resources (whenever possible).

Vulkan separates the concept of a resource from the physical memory which backs it. This makes it possible to reuse the same physical memory for multiple different resources at different points in the rendering pipeline.

The ability to alias memory resources can be used to reduce the total memory footprint of the application by recycling the same physical memory for multiple uses at different points in a frame creation.

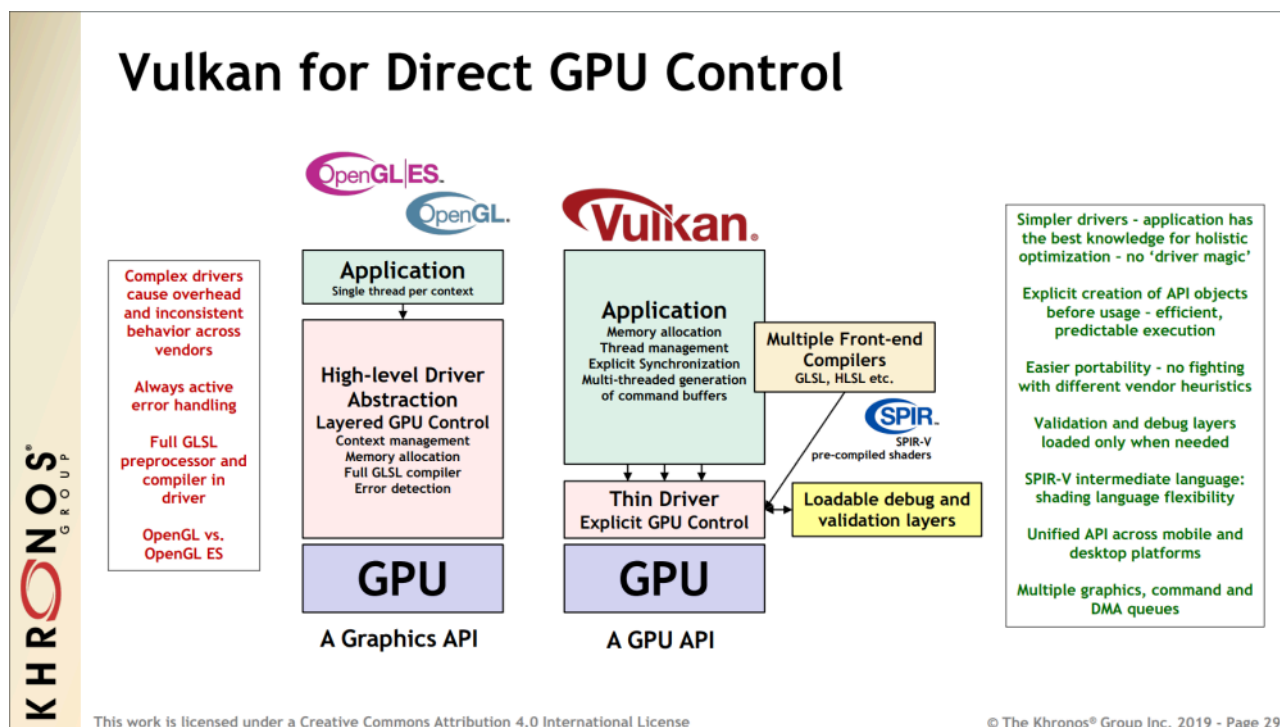
1.2.2 - Disadvantages

One of the major drawbacks of Vulkan is that it places a significant amount of responsibility on the application, including memory allocation and resource synchronization. While this provides a greater level of control and customization, it also increases the risk of suboptimal application behavior, resulting in decreased performance.

1.2.3 - Conclusions

The primary benefits of using Vulkan are associated with a lower CPU overhead, which is attributed to the minimal level of abstraction provided by the Vulkan drivers. It is important to note that Vulkan does not necessarily guarantee a performance enhancement. The GPU hardware remains the same, and the rendering capabilities made available by Vulkan are similar to those offered by OpenGL. If your application's performance is restricted by GPU rendering speed, it is improbable that Vulkan will result in improved performance.

However, If you're looking for an API that better exposes how GPUs really work, with an high degree of control over resource management, and available for cross-platform (both Desktop and mobile), multi-threaded applications, then you should definitely consider using Vulkan.



Applications which use Vulkan well can take advantage of multi-threading, and benefit from reduced CPU load and memory footprint, as well as smoother rendering with fewer hitches caused by thicker driver abstractions.

In the image above you can also see that OpenGL includes a complete shader compiler in the driver, which increases the level of abstraction and complexity of the driver, leading to additional CPU overhead.

The predictable execution specified in the image refers to the fact that, by using Vulkan, we can set up the rendering work ahead of time, which allows for better holistic (overall) optimization by the driver.

Vulkan is an explicit API with a thin driver abstraction, making it easier to develop and port to different platforms. This also means that vendor implementations don't have to rely on advanced heuristics to guess what the application is trying to do.

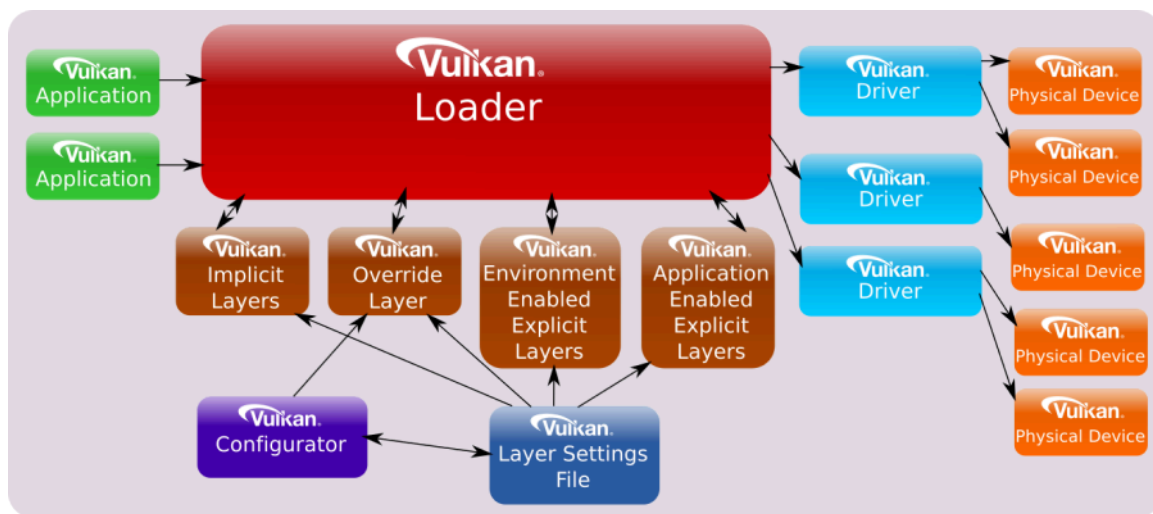
SPIR-V is a binary intermediate representation for shader programs. In Vulkan, you can still write the shader code for programmable stages in a high-level shading language such as GLSL or HLSL, but a SPIR-V binary representation is needed when setting the pipeline state. Since it is pre-compiled, the driver can easily convert SPIR-V binary blob to shader machine (GPU) code.

1.3 - Important Vulkan concepts and components

Vulkan defines a layered API in a layered architecture.

The components of the layered architecture are:

- Vulkan Applications
- The Vulkan Loader
- Vulkan Layers
- Drivers
- VkConfig



These components interact with the Vulkan API by using, implementing, extending, or intercepting its functions. The Vulkan API is designed in a layered manner, which means that applications can use a subset of its functions that all Vulkan implementations have to support as part of the Vulkan core specification. Then, applications can also use additional layers above the core one for debugging, validation, and other purposes.

1.3.1 - Vulkan Applications

To use the functionality specified in the Vulkan specification, applications use the function prototypes included in the header files provided by the Khronos Group. These files can be found in the Vulkan-Headers repository on GitHub, or in the Vulkan SDK. Vulkan functions are somewhat defined in the Vulkan Loader module (more on this shortly), which is installed via driver update packages or the Vulkan SDK. Alternatively, the Vulkan loader can be compiled from source code. Anyway, linking the Vulkan loader module is required for Vulkan applications.

Generally, a Vulkan application uses the Vulkan API to perform the following sequence of tasks:

- Initialize the hardware and software (to detect the available devices and let the loader start its job).
- Create a presentation surface using a Window System Integration (WSI) extension.
- Set the pipeline state.
- Create resources to bind to the stages of the pipeline using descriptors.
- Record commands in command buffers in a render loop.
- Submit the command buffers to GPU queues for processing.

We will cover each of the above issues in detail, both in this tutorial and in upcoming ones.

Direct Exports

The Vulkan loader module exports all core entry-points and some additional platform-specific functions (more on this shortly). Therefore, most of the time we can simply link the loader and call the Vulkan functions using the prototypes declared in the related header files.

On Linux, using GCC, we can link the Vulkan loader with the following command, which links "[libvulkan.so](#)". This file is a symbolic link to "libvulkan.so.1", which in turn is a symbolic link to the latest installed Vulkan loader (e.g., "libvulkan.so.1.0.42.0", which is the actual shared library).

```
g++ main.cpp -o main.out -lvulkan
```

On Windows, we can implicitly link the Vulkan loader with the help of the related import library "vulkan-1.lib". This file includes a table where each element stores the name of a function and the module where to find the related definition. An import library simply tells the linker not to panic if it cannot find a definition, because the Windows loader will be able to dynamically resolve\fix it at run-time by loading the related shared module ("vulkan-1.dll", which is just a copy to the latest Vulkan loader installed (e.g. "vulkan-1-999-0-0-0.dll").

Dynamic linking

Alternatively, we can load the Vulkan Loader module at runtime with **dlopen** (Linux) or **LoadLibrary** (Windows). Then, we only need to query the addresses of **vkGetInstanceProcAddr** and **vkGetDeviceProcAddr** with **dlsym** (Linux) or **GetProcAddress** (Windows). At that point, with **vkGetInstanceProcAddr** and **vkGetDeviceProcAddr** we can query the addresses of other Vulkan functions.

In the next section we will see what's the difference between using direct exports and dynamic linking, as well as the difference between **vkGetInstanceProcAddr** and **vkGetDeviceProcAddr**.

Static linking was possible in the past, but it is no longer an option now.

1.3.2 - Vulkan Loader

It was previously stated that the loader somewhat define Vulkan functions, but what exactly does it means? Well, you can think of the definitions in the loader as trampolines that redirect application calls to the appropriate implementations in the graphics drivers, passing through an arbitrary number of layers.

The loader can insert optional layers between the application and the drivers, which can offer specialized functionality. The loader is essential for ensuring that Vulkan functions are dispatched to the correct set of layers and drivers. By inserting layers into a call-chain, the loader can enable them to process Vulkan functions before the driver is invoked.

To understand how the loader directs API calls, it is necessary to have a basic knowledge of certain key concepts. The most important one is that many objects and functions in Vulkan can be separated into two groups:

- Instance-specific
- Device-specific

"Instance-specific" refers to functions or objects that can be used to provide system-level information and functionality (i.e., something that can be applied to all available GPUs, and their drivers, installed on a user's system).

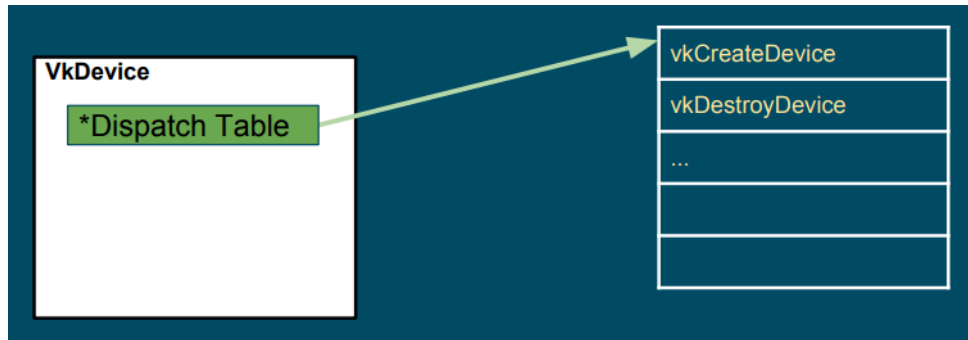
"Device-specific" means something which applies to a particular physical device on the user's system.

Examples of instance objects are **VkInstance** and **VkPhysicalDevice**. We will use these objects as interfaces to call instance

functions. Instance functions are functions that take either an instance object as their first parameter, or nothing at all.

Examples of device objects are **VkDevice**, **VkQueue**, and **VkCommandBuffer**. We will use these objects as interfaces to call device functions. Device functions are functions that take a device object as their first parameter.

VkInstance, **VkPhysicalDevice**, **VkDevice**, **VkQueue**, and **VkCommandBuffer** are called dispatchable objects, which means they contain a pointer to a dispatch table. A dispatch table is an array of function pointers (including core and possibly extension functions) used to step to the next entity in a call chain. An entity could be the loader, a layer or a driver (more on this shortly).

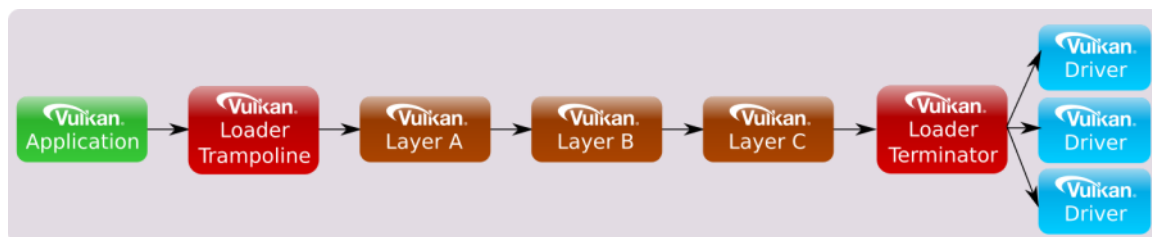


The loader maintains two types of dispatch tables: instance specific, created during the call to **vkCreateInstance** (which creates a **VkInstance**), and device specific, created during the call to **vkCreateDevice** (which creates a **VkDevice**). Thus, the loader builds an instance call chain for each **VkInstance** that is created, and a device call chain for each **VkDevice** that is created (that is, the actual instance function called depends on the **VkInstance** object passed as its first parameter, which points to its own dispatch table; the same applies to device functions). The pointer to the dispatch table stored in **VkPhysicalDevice**, **VkQueue**, and **VkCommandBuffer** is a duplicate of the pointer from the existing parent object of the same level (**VkInstance** versus **VkDevice**). For example, the dispatch table pointer in a **VkPhysicalDevice** object is a duplicate of the pointer in the corresponding **VkInstance** object, while the dispatch table pointer in a **VkQueue** or **VkCommandBuffer** object is a duplicate of the pointer in the corresponding **VkDevice** object.

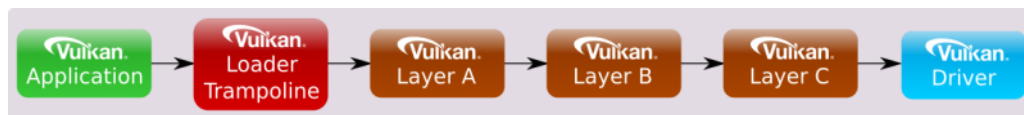
At the time of calling **vkCreateInstance** or **vkCreateDevice**, the application and the system can each specify optional layers to be included. The loader will initialize the specified layers to create a call chain for each Vulkan function, and each entry of the dispatch table in the first entity of the call chain will point to the corresponding entry in the next entity of the same chain. Further information will be provided in the next section.

After an application makes a Vulkan function call, this is usually directed to a trampoline function in the loader. Trampoline functions are small, simple functions that jump to the appropriate dispatch table entry, which is located in the loader, layer, or device driver, depending on the pointer to the dispatch table of the dispatchable object passed as the first parameter. Then, the first dispatch table entry can point to the corresponding entry in the next entity in the call chain (layer or device driver), which calls the corresponding entry in the next entity, and so on.

Additionally, for functions in the instance call chain, the loader has an additional function called a terminator. This function is called after all enabled layers to marshal the appropriate information to all available drivers.



Device call chains are generally simpler because they deal with only a single device. As a result, the specific driver that exposes this device can always be the terminator of the chain.



The functions exported by the loader are trampolines to function pointers stored in the dispatch tables maintained by the loader itself.

However, it is possible to set up a custom dispatch table by querying instance functions using **vkGetInstanceProcAddr**, and device functions using **vkGetDeviceProcAddr**. The main difference between the two is that **vkGetInstanceProcAddr** may return the address of the same trampoline function exported by the loader, whereas **vkGetDeviceProcAddr** returns the actual function pointer stored in the dispatch table of the loader, whenever possible. This can optimize the call chain even further by removing the loader altogether in most scenarios:

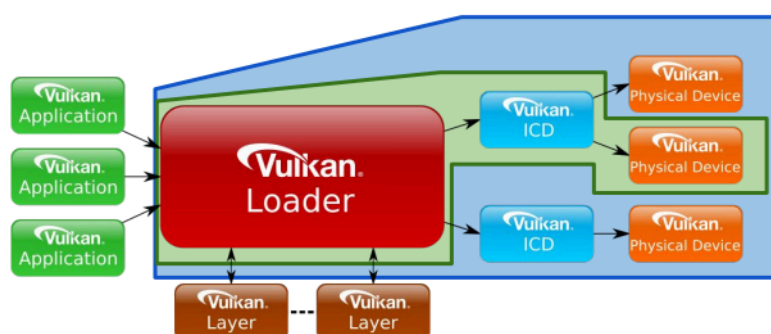


In this tutorial series, we will use the function prototypes declared in the header files provided by the Khronos Group to directly call the functions exported by the loader. However, we will also use **vkGetInstanceProcAddr** and **vkGetDeviceProcAddr** to query the addresses of instance and device functions exposed but not exported by the loader.

At this point, it can be explained why a physical device (**vkPhysicalDevice**) is an instance object (instead of a device object). A physical device can be considered a stand-alone object representing a generic GPU installed on the user's system. On the other hand, a device object is considered as a logical accessor to a particular physical device through a particular ICD (driver). Indeed, **VkDevice** objects are also called logical device objects. Multiple logical devices can be created from the same physical device, each with its own state and resources independent of other logical devices.

Instance: High-level construct (similar, but not the same, as a GL Context)
Works with all ICDs and Physical Devices
Includes: **VkInstance** and **VkPhysicalDevice**

Device: Logical accessor to a particular Physical Device (through a particular ICD)
Includes: **VkDevice**, **VkQueue**, **VkCmdBuffer** and other objects derived from these



Extensions

Extensions in Vulkan refer to additional functionalities that can be provided by a layer, the loader, or a driver to extend the core Vulkan functionality. They can introduce new features that may be experimental, specific to a particular platform or device, or come with a performance cost. Consequently, extensions are not enabled by default and must be explicitly enabled by an application before they can be used. This approach ensures that applications only pay for the functionalities they need, and that the use of extensions is intentional, which helps improve performance and portability.

Extensions can introduce new functions, enums, structs, or feature bits to enhance the core functionality of Vulkan. While the related definitions are included in the Vulkan Headers by default, using them without enabling the corresponding extensions can result in undefined behavior. Therefore, to ensure correct and predictable behavior, applications must explicitly enable the required extensions before using any extended Vulkan functionality.

Vulkan extensions are categorized based on the type of functions they offer. This categorization results in extensions being divided into instance or device extensions where the majority of functions in the extension correspond to the respective type. For instance, an "instance extension" mainly comprises "instance functions," which accept instance objects as their first parameter to introduce new general functionality accessible to the whole system. On the other hand, a "device extension" mainly includes "device functions," which accept device objects as their first parameter to provide new functionality to a particular device.

Usually, Vulkan extensions include an author prefix or suffix modifier to every structure, enumeration entry, command entry-point, or define that is associated with it. For example, the "KHR" prefix indicates Khronos-authored extensions, and it can also be found on structures, enumeration entries, and commands associated with those extensions. The "EXT" suffix is used for multi-company authored extensions, while "AMD," "ARM," and "NV" are used for extensions authored by AMD, ARM, and Nvidia, respectively.

Windows System Integration (WSI) extensions are well known Vulkan extensions targeting a particular Windowing system and designed to interface between the Windowing system and Vulkan. Some WSI extensions are valid for all windowing system, but others are particular to a given execution environment. The loader only enables and directly exports those WSI extensions that are appropriate to the current environment (Windows, Linux, etc.).

It's worth noting that although the loader can export entry points directly for these extensions, an application can use them only under certain conditions:

- At least one physical device must support the extension(s)
- The application must use such a physical device when creating a logical device
- The application must request the extension(s) be enabled while creating the instance or logical device (this depends on whether or not the given extension works with an instance or a device)

Only then the WSI extension can be properly used in a Vulkan program.

Showing the results of rendering operations on the screen is a fundamental operation. So, why is it provided as an extension instead of a core functionality? Well, there are at least two reasons:

- Vulkan is a platform-agnostic API, and each platform may use different window systems that interact with the operating system in different ways. Therefore, Vulkan cannot provide a generic interface that works for all window systems on all platforms.
- A Vulkan application is not obligated to show the result of graphics operations on the screen. Instead, the specification states that we must store the result on a render target (usually a texture). Then, we can save the render target to a disk file or display it to the user on the screen. Additionally, rendering to a render target is a platform-specific operation that depends on the window system.

As Vulkan can be easily expanded, there may be extensions that the loader is not aware of, but that it still needs to create. In such cases, if the extension is a device extension, the loader will pass the unknown entry-point down the device call chain until it reaches the appropriate driver entry-points (more on this in the next section). The same will happen if the extension is an instance extension that takes a physical device as its first parameter.

However, for all other instance extensions the loader will fail to dispatch the call. The reason is that the loader needs information on the physical devices that support the extension (it can't simply pass the call to all the device drivers available in the system in the hope that the extension is supported by all of them).

At times, enabling an extension may also require enabling an optional device feature. Vulkan features define functionality that may not be supported on all physical devices. If a device advertises support for a feature, it still needs to be enabled (similar to an extension). However,

once enabled, the feature becomes core functionality instead of an extension.

the difference between features and extensions is that features refer to functionality that may be supported by a physical device, whereas extensions refer to functionality that is either implemented by a Vulkan implementation or can be injected into the call chains (instance or device) by loading the appropriate Vulkan layers.

1.3.3 - Vulkan Layers

Applications desiring Vulkan functionality beyond what Vulkan drivers on their system already expose, may use various layers to augment the API. Each layer can choose to hook (intercept) Vulkan functions to be inspected, augmented or simply ignored. However, any function a layer does not hook is skipped for that layer, and the control flow will continue on to the next layer, or driver. As a result, a layer has the option to intercept all Vulkan functions, or only a specific subset that it finds relevant.

Vulkan layers allow applications to use additional functionalities, mainly for development purposes. For example:

- Validating API usage
- Tracing API calls
- Debugging aids
- Profiling
- Overlay

Layers are unable to introduce new Vulkan core API entry-points that are not already available in the `vulkan.h` header file. However, they can provide implementations of extensions that introduce additional entry-points that are beyond what is available without those layers. These extra extension entry-points can be accessed through specific extension header files.

Layers are packaged as shared libraries the loader can find and dynamically load using some manifest files (data files in JSON format) installed by the Vulkan SDK. Since layers are optional and loaded dynamically, they can be enabled or disabled based on requirements. During application development and debugging, enabling certain layers can help ensure the proper usage of the Vulkan API. However, when releasing the application, those layers become redundant and can be disabled, resulting in a faster performance of the application.

Layers can be classified into two categories:

- Implicit Layers
- Explicit Layers

Explicit layers need to be explicitly enabled by an application.

Implicit layers are enabled by default, unless an additional manual enable step is required. These layers have an additional requirement compared to explicit layers, as they need to be disabled by an environmental variable. This is because implicit layers are not visible to the application and may potentially create issues. That way, users have the option to disable an implicit layer if it causes problems.

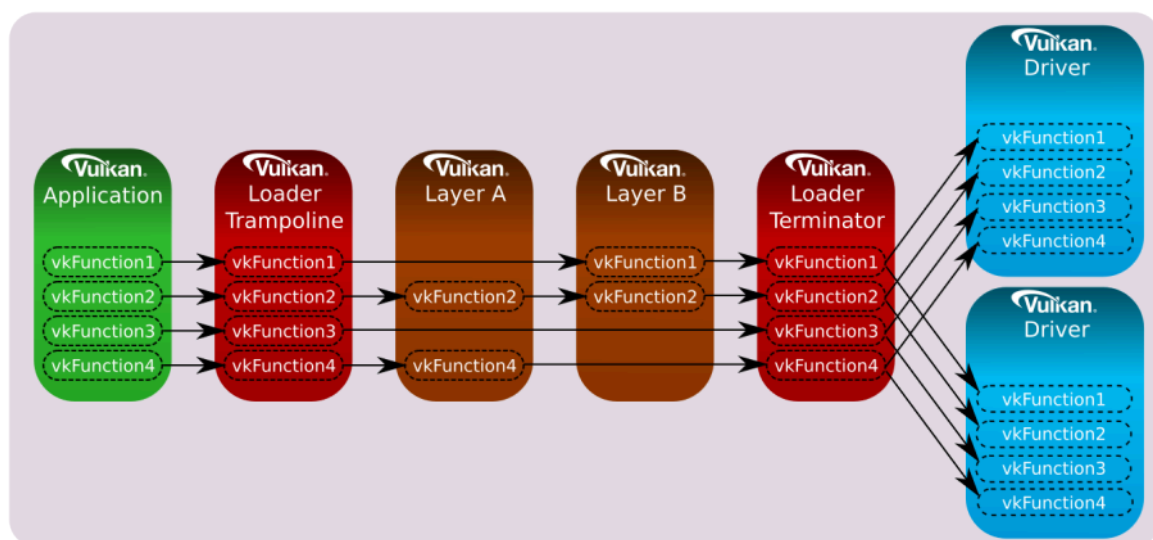
On any system, the loader searches specific areas for information about the layers that it can load, both implicitly and explicitly. That is why it is crucial to install the Vulkan SDK as it configures the system by generating the necessary keys (in the Windows registry) and files that contain information about where to locate the layers, or more precisely, the shared libraries that implement them.

Please note that layers are typically not enabled when releasing an application. Hence, users do not require installing the Vulkan SDK. They only need to have the Vulkan Loader and drivers installed through the vendor driver update packages.

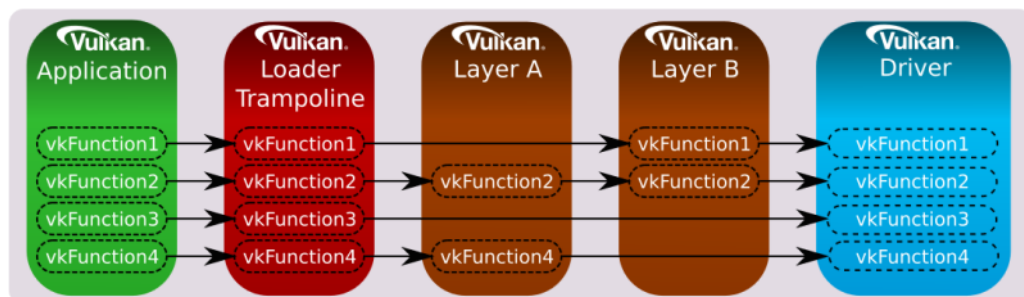
A layer has the capability to intercept instance functions, device functions, or both. To intercept instance functions, a layer must participate in the instance call chain. To intercept device functions, a layer must participate in the device call chain. However, as mentioned earlier, a layer is not required to intercept all instance or device functions. It can instead choose to intercept only a subset of those functions.

When a layer intercepts a particular Vulkan function, it generally calls down the instance or device call chain as required. The loader, along with all layer libraries that participate in a call chain, work together to ensure the correct sequencing of calls from one entity to the next. This coordinated effort for call chain sequencing is known as distributed dispatch. Under distributed dispatch, each layer is responsible for correctly calling the subsequent entity in the call chain. Consequently, a dispatch mechanism is necessary for all Vulkan functions that a layer intercepts.

For example, if only specific instance functions were intercepted by the enabled layers, then the instance call chain would appear as follows:



Likewise, if only specific device functions were intercepted by the enabled layers, then the device call chain would appear as follows:



The loader is responsible for dispatching all core and instance extension functions to the first entity in the call chain.

If you're curious about the dispatch mechanism that allows a layer to know the next entity in the call chain, along with the address of an intercepted Vulkan function in the next entity. Well, long story short, the loader is aware of all the enabled layers because this information is required before calling **vkCreateInstance**. Additionally, each layer must intercept **vkCreateInstance**, and provide implementations for **vkGetInstanceProcAddr** and **vkGetDeviceProcAddr**, which return the addresses of intercepted functions in (the shared module\library of) the layer. These two functions must also be exported symbols in (the shader module\library of) the layer, allowing the loader to get their addresses to create a linked list where each element stores the addresses of both these functions for each layer in the call chain.

When an application calls **vkCreateInstance**, the corresponding function in the loader is invoked. During initialization, the loader creates

the linked list and stores it in a field of a structure passed as an input parameter to **vkCreateInstance**. Then, the loader uses the first element of the linked list to retrieve the address of **vkGetInstanceProcAddr** in the first entity of the call chain, and uses this address to build its own dispatch table, where each element is the address of a function intercepted in the first entity. At that point, the loader pops the first element off the front of the linked list and calls **vkCreateInstance** into the first entity, passing the same arguments it received in input by the application. The implementation of **vkCreateInstance** in the first entity performs the same task as the loader: it uses the first element in the linked list to retrieve the address of **vkGetInstanceProcAddr** in the second entity of the call chain, and uses this address to build its own dispatch table, where each element is the address of a function intercepted in the second entity. At that point the first entity pops off the front of the linked list, and calls the **vkCreateInstance** of the third entity. And so on.

Now, what happens if a layer doesn't intercept a function? In other words, what address should return **vkGetInstanceProcAddr** for non-intercepted functions? Well, in that case a layer can use the address of **vkGetInstanceProcAddr** in the next entity to forward the call down the call chain. Observe that **vkCreateInstance** is the first function invoked in a layer, so that the address of **vkGetInstanceProcAddr** in the next entity is always available for a layer when other Vulkan functions are invoked in the call chain.

Implementing **vkGetDeviceProcAddr** and **vkCreateDevice** in a layer follows almost the same pattern as the instance versions, with the layers on a different call chain, although - the device call chain instead of the instance call chain).

Additional information can be found in [2].

The loader supports filter environment variables that can forcibly enable and disable the layers that are already detected by the loader based on default search paths and other environment variables. The filter variables are compared against the layer name provided in the layer's manifest file.

Meta-layers consist of a collection of layers that are arranged in a specific order to ensure proper interaction between them. This approach was initially used to group Vulkan Validation layers together in a specific order to prevent conflicts. The reason for this was that validation layers was divided into multiple component layers. However, the new validation layer pull everything into a single layer, thereby eliminating the need for meta-layers.

1.3.4 - Vulkan Configurator

VkConfig is a tool included in the Vulkan SDK that uses meta-layers to group layers together based on user's preferences. It can be used to find layers, enable or disable them, change layer settings, and other useful features.

VkConfig creates three configuration files, two of which are intended to operate in conjunction with the Vulkan loader and layers. These files are:

- The Vulkan Override Layer
- The Vulkan Layer Settings File
- VkConfig Configuration Settings

The "Override Layer" is a special implicit meta-layer created by VkConfig, and available by default when the tool is running. Once VkConfig exits, the override layer is removed, and the system should return to standard Vulkan behavior. Whenever the override layer is present, the loader will pull it into the layer call stack along with all (implicit and explicit) layers to load. This layer forces the loading of the desired layers that were enabled inside of VkConfig, and disables those layers that were intentionally disabled (including implicit layers).

The Vulkan Layer Settings file can be used to tell each enabled layer which settings to use. Per-layer settings are loaded by each layer library and stored in the **vk_layer_settings.txt** file. This file is either located next to the Vulkan application executable or set globally and applied to all Vulkan applications thanks to Vulkan Configurator.

The VkConfig Configuration Settings file stores the application settings for VkConfig.

The Vulkan Configurator does not make any system-wide changes to a system, but it does make user-specific changes.

1.3.5 - Vulkan Drivers

A module that implements the Vulkan specification, either through supporting a physical hardware device directly, converting Vulkan commands into native graphics commands (like **MoltenVK** for macOS and iOS), or simulating Vulkan through software, is considered "a driver". The most common type of driver is the Installable Client Driver (or ICD). These are drivers that are provided by hardware vendors to interact with the hardware they provide.

The loader is responsible for discovering available Vulkan drivers on the system. Given a list of available drivers, the loader can enumerate all the available physical devices and provide this information to applications.

Vulkan allows multiple ICDs, each supporting one or more devices. Each of these devices is represented by a **VkPhysicalDevice** object. The loader is responsible for discovering available Vulkan ICDs on the system.